Prompt 1

ion

Actúa como tutor de Python para Ciencia de Datos. Explícame detalladamente en qué se diferencian los distintos tipos de funciones en Python (por ejemplo: built-in, definidas por el usuario con def, lambda/anónimas, métodos (instancia/clase/estático), generadoras con yield y funciones async). Para cada tipo: (1) definición breve, (2) un ejemplo corto en Python, y (3) un caso real de cuándo usarla.

¡Claro! Me encanta ayudar a profundizar en Python, especialmente cuando se trata de Ciencia de Datos. Entender los tipos de funciones es clave para escribir código eficiente y elegante. Vamos a desglosar cada una:

1. Funciones Built-in (Integradas)

- **Definición:** Son funciones que vienen preinstaladas en Python, listas para usar sin necesidad de importarlas. Son la base de muchas operaciones comunes.
- **Ejemplo:**

```
numeros = [10, 20, 5, 30, 15]
max_numero = max(numeros)
print(max_numero) # Salida: 30
```

3 lines (3 loc) · 98 B

- **Caso de Uso en Ciencia de Datos:** Son omnipresentes. Desde calcular la suma de una columna (`sum()`), encontrar el valor máximo o mínimo (`max()`, `min()`), ordenar listas (`sorted()`), convertir tipos de datos (`int()`, `str()`), hasta leer archivos (`open()`). Son herramientas fundamentales para la manipulación y preparación inicial de datos.

2. Funciones Definidas por el Usuario (`def`)

Pregunta lo que quieras...

Prompt 2

Actúa como ingeniero/a de software. ¿Bajo qué criterios técnicos es preferible que una función definida por el usuario reciba parámetros o no? Incluye aspectos como acoplamiento, pureza, dependencias, configuración, testabilidad y reutilización. Además, ¿en qué escenarios es mejor retornar un valor frente a solo ejecutar una acción con efectos secundarios? Da ejemplos concretos en Python.

¡Claro! Como ingeniero de software, te diré que la decisión de si una función debe recibir parámetros o no, y si debe retornar un valor o ejecutar una acción, es fundamental para escribir código robusto, mantenible y escalable. Vamos a desglosarlo.

Criterios técnicos para recibir parámetros en una función

La regla general es que, si una función necesita datos externos para realizar su tarea, esos datos deben pasarse como parámetros. Esto se basa en varios principios clave:

- **Acoplamiento (Coupling):**
  - **Con parámetros:** Reduce el acoplamiento. La función no necesita saber de dónde vienen los datos; solo sabe que los recibe. Esto la hace más independiente.
  - **Sin parámetros (dependiendo de estado global o de clase):** Aumenta el acoplamiento. Si una función lee directamente variables globales o atributos de una clase sin recibirlos explícitamente, se acopla fuertemente a ese estado. Cambiar el estado en un lugar puede romper la función en otro, incluso si no parece directamente relacionado.
  - **Ejemplo:**
    - **Alto acoplamiento:**

```
config_data = {"threshold": 100} # Variable global
def process_item(item):
    if item.value > config_data["threshold"]:
        print(f"Item {item.id} sup el umbral.")
```

Pregunta lo que quieras...

Rápido

Prompt 3

Compara funciones definidas por el usuario (UDF con def) vs funciones lambda en Python. ¿En qué situaciones específicas es preferible usar def en lugar de lambda? Explica el impacto en legibilidad, depuración, documentación, pruebas y reutilización. Incluye un ejemplo con lambda y su refactor a def (y explica por qué mejora).

¡Claro! Vamos a desglosar las diferencias entre las funciones definidas con `def` (UDFs) y las funciones `lambda` en Python, y por qué a menudo es mejor usar `def` en situaciones más complejas.

Las funciones `lambda` en Python son pequeñas funciones anónimas de una sola expresión, mientras que las funciones `def` pueden contener múltiples expresiones y declaraciones.

Aquí te explico los puntos clave donde `def` se lleva la ventaja:

- **Legibilidad:** Las funciones `def` permiten nombres descriptivos, lo que facilita entender su propósito con solo ver su nombre. También pueden tener varias líneas de código y comentarios para explicar su lógica. Las `lambda` son concisas pero, al ser anónimas y de una sola línea, pueden volverse difíciles de descifrar si la lógica es un poco intrincada.
- **Depuración:** Si tu función `lambda` tiene un error, puede ser complicado rastrearlo. No tienen nombres, lo que hace que los rastreos de pila sean menos informativos. En cambio, con `def`, el nombre de la función y la posibilidad de añadir `print` o usar un depurador paso a paso hacen que encontrar y solucionar problemas sea mucho más sencillo.
- **Documentación:** Las funciones `def` pueden incluir "docstrings" (cadenas de documentación) que describen su propósito, argumentos y lo que retornan. Esto es esencial para que otros (y tu yo futuro) entiendan cómo usar la función. Las `lambda` no soportan docstrings.
- **Pruebas:** Es más fácil escribir pruebas unitarias para funciones `def` porque tienen nombres claros y pueden ser importadas y probadas individualmente. Para una `lambda` más compleja, podría ser necesario aislarla de su contexto para probarla, lo cual añade fricción.
- **Reutilización:** Una vez definida con `def`, una funci uede ser llamada múltiples veces en diferentes partes de tu código. Las `lambda` suelen ser de un solo uso, definidas en el lugar donde se necesitan, lo

Pregunta lo que quieras...

Rápido

Reflexión

Siento que ahora tengo una comprensión mucho mejor de qué son las funciones, cómo construirlas y cómo usarlas de manera efectiva en mi propio código. Usar a Meta AI para este tipo de aprendizaje es como tener un asistente de programación siempre disponible, dispuesto a explicarte cualquier cosa y a darte ejemplos prácticos.